

This algorithm also has the constraint that the array must be sorted.

★ The time complexity of this algorithm is $O(\log_3 N)$ where 'N' is the size of the array.

3. Ternary Search: A search algorithm that is used to find the position of a target value within a sorted array. It operates on the principle of dividing the array into three parts instead of two, as in binary search.

The basic idea ~~of~~ is to narrow down the search space by comparing the target value with elements at two points that divide the array into three equal parts.

• ~~$$\text{mid1} = 1 + \frac{(e-1)}{3}$$~~

• ~~$$\text{mid2} = e - \frac{(e-1)}{3}$$~~

• When to use Ternary Search

- When you have a large sorted array or list and need to find the position of a specific value.
- When you need to find the maximum or minimum value of a function.
- When you need to find the bitonic point in a bitonic sequence.
- When you have to evaluate a quadratic expression.

• Working of Tertiary Search

Works similarly to a binary search, with the distinction of reducing time complexity by dividing the array into three parts instead of two.

① Initialization

- Set two pointers, left and right, initially pointing to the first and last elements of our search space.

② Divide the search space

- Calculate two midpoints, mid1 and mid2, dividing the current search space into three roughly equal parts.

$$\bullet \text{ mid1} = \text{left} + \frac{(\text{right} - \text{left})}{3}$$

$$\bullet \text{ mid2} = \text{right} - \frac{(\text{right} - \text{left})}{3}$$

- The array is now effectively divided into $[\text{left}, \text{mid1}]$, $(\text{mid1}, \text{mid2})$, and $[\text{mid2}, \text{right}]$.

③ Comparison with Target

- If the target is equal to the element

- at mid_1 or mid_2 , the search is successful, and the index is returned.
- If the target is less than the element at mid_1 , update the right pointer to $\text{mid}_1 - 1$.
- If the target is greater than the element at mid_2 , update the left pointer to $\text{mid}_2 + 1$.
- If the target is between the elements at mid_1 and mid_2 , update the left pointer to $\text{mid}_1 + 1$ and the right pointer to $\text{mid}_2 - 1$.

④ Repeat or Conclude

- Repeat the process with the reduced search space until the target is found or the search space becomes empty.
- If the search space is empty and the target is not found, return a value indicating that the target is not present in the array.

Target is not found ☹️

Q. Search for 8 in the array:
 $arr[] = \{1, 4, 6, 8, 10, 12, 15\}$

0	1	2	3	4	5	6
1	4	6	8	10	12	15

 $l = 0$
 $mid1$
 $mid2$
 $e = 6$
 $target = 8$

$$mid1 = l + \frac{(e - l)}{3} = 0 + \frac{(6 - 0)}{3} = 2$$

$$mid2 = e - \frac{(e - l)}{3} = 6 - \frac{(6 - 0)}{3} = 4$$

$8 > 6^{(mid1)}$ and $8 < 10^{(mid2)}$, so search in

$\Rightarrow [mid1 + 1, mid2 - 1]$

$\Rightarrow [3, 3]$

$\Rightarrow$ Target 8 is found at index 3.

①

Iterative Approach of Ternary Search

```
#include <iostream>
using namespace std;
```

```
// Function to perform Ternary Search
```

```
int ternarySearch(int l, int r, int key,
int arr[]) {
```

```
while (r >= l) {
```

```
// Find the mid1 and mid2
```

```
int mid1 = l + (r - l) / 3;
```

```
int mid2 = r - (r - l) / 3;
```

```
// Check if key is present at
```

```
// any mid
```

```
if (arr[mid1] == key) {
```

```
return mid1;
```

```
if (arr[mid2] == key) {
```

```
return mid2;
```

```
// Since key is not present
```

```
// at mid, check in which
```

```
// region it is present then
```

```
// repeat the Search operation
```

```
// in that region
```

if (key < arr[mid1]) {

// The key lies in between
// mid1 and mid2
e = mid1 - 1;

}

else if (key > arr[mid2]) {

// The key lies in between
// mid2 and e
l = mid2 + 1;

}

else {

// The key lies in between
// mid1 and mid2
l = mid1 + 1;
e = mid2 - 1;

}

// Key not found
return -1;

}

// Driver code
int main() {

int l, e, p, key;

// Get the array
 // Get the array if not sorted
 int arr[] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};

// Starting index
 l = 0;

// end element index
 r = 9;

// Checking for 5

// Key to be searched in the array
 Key = 5;

// Search the Key using Binary Search
 p = BinarySearch(l, r, Key, arr);

// Print the result
 cout << "Index of " << Key << " is "
 << p << endl;

// Index of 5 is 4

Time Complexity: $O(2 * \log_2 n)$, where n is the size of the array

Auxiliary Space: $O(1)$

2

• Recursive Approach of Ternary Search

// C++ program to illustrate
// recursive approach to ternary search

```
#include <iostream>
#include <bits/stdc++.h>
using namespace std;
```

// Function to perform Ternary Search
int ternarySearch(int l, int e, int Key,
int arr[])

```
{
    if (e <= 1) {
```

```
        // Find the mid1 and mid2
        int mid1 = l + (e - l) / 3;
        int mid2 = e - (e - l) / 3;
```

```
        // Check if Key is present at any mid
        if (arr[mid1] == Key) {
            return mid1;
        }
        if (arr[mid2] == Key) {
            return mid2;
        }
```

```
        // Since Key is not present at mid,
        // check in which region it is present
        // then repeat the search operation
        // in that region
```

```
if (Key < arr[mid1]) {
```

```
// The Key lies in between  
// 1 and mid1
```

```
return binarySearch(l, r,  
mid1-1, Key, arr);
```

```
}
```

```
else if (Key > arr[mid2]) {
```

```
// The Key lies in between mid2  
// and r
```

```
return binarySearch(mid2+1, r,  
Key, arr);
```

```
}
```

```
else {
```

```
// The Key lies in between  
// mid1 and mid2
```

```
return binarySearch(mid1+1, r,  
mid2-1, Key, arr);
```

```
}
```

```
// Key not found  
return -1;
```

```
}
```

```
// Driver code
```

```
int main() {
```

```
int main() {
```

```
int l, r, p, Key;
```

// Get the array

// Get the array if not sorted

int arr[] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};

// Starting index

l = 0;

// end element index

e = 9;

// Checking if 5

// Key to be searched in the array

key = 5;

// Search the key using ternary search

p = ternarySearch(l, e, key, arr);

// Print the result

cout << "Index of " << key
<< " is " << p << endl;

}

~~// Check~~

// Index of 5 is 4

Time Complexity: $O(2^{\log_3 n})$

Auxiliary Space: $O(\log_3 n)$

• Complexity Analysis of Ternary Search

• Time Complexity:

• Best Case: $\Omega(1)$

• Average Case: $O(\log_3 N)$

• Worst Case: $O(\log_3 N)$

• Auxiliary Space: $O(1)$

• Binary Search vs Ternary Search

★ ~~Time~~ The time complexity of binary search is less than the ternary search as the number of comparisons in ternary search is much more than binary search.

Binary Search is used to find the maxima/minima of monotonic functions whereas ternary search is used to find the maxima/minima of unimodal functions.

Note: We can also use ternary search for monotonic functions but the time complexity will be higher as compared to binary search.

• Advantages

1. Ternary Search can find maxima/minima for unimodal functions, where binary search is not applicable.
2. Ternary search has a time complexity of $O(2 * \log_3 n)$, which is more efficient than linear search and comparable to binary search.
3. Fits well with optimization problems.

• Disadvantages

1. Ternary Search is only applicable to sorted lists or arrays, and cannot be used on unsorted or non-linear data sets.
2. Ternary Search takes more time to find maxima/minima of monotonic functions as compared to Binary Search.

The array to be searched must be sorted for Ternary Search to work correctly.

[2 | 2 | 5 | 5 | 7 | 7] = []

↑

↑

low

high